第二章 词法分析

> Important
>
> **重点：**
>
> - 词法分析器功能
> - 词法分析器实现
> - 词法分析的错误恢复（区分什么样的错误能在词法分析阶段发现，包括后面语法分析阶段各能发现什么样的错误等）
> - 根据正规式写出语言描述，根据语言描述能写出正规式/正规定义
> - NFA、DFA定义、区别、表示方法
> - 正规式 → NFA → DFA → 最简DFA

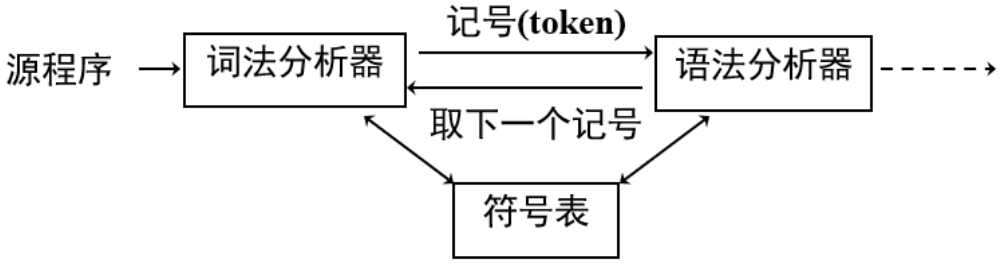
词法分析器概述

1、核心功能

- 找出源程序字符流中有意义的词法单元（lexemes），并为每一个词法单元生成记号（token）输出。
- ❗ **剥去注释和空白**（制表符、回车符、空格等分隔符）
- 将各阶段产生的错误信息和源程序联系起来
- 宏处理（可选）

2、与语法分析器的关系

- 主从关系：**语法分析器为主导**，词法分析器为其子程序
- 工作模式：交错进行（语法分析器要求"取下一个记号"时，词法分析器才工作）



3、构造方法

- 手工构造：用状态转换图描述记号结构 → 手工翻译成识别程序
- 自动生成：正规式 → NFA → DFA → 最简DFA → （识别器）词法分析器

词法记号

词法分析器产生的**最终结果**，也是下一阶段（语法分析器）处理的**基本输入单元。**

三个核心概念

```
if (x1 > 100)
```

我们来拆：

Lexeme	Pattern	Token
`if`	标识符模式（形式上符合）	`<if, _>`
`(`	左括号模式	`<lparen, _>`
`x1`	`letter(letter|digit)*`	`<id, x1>`
`>`	大于号模式	`<gt, _>`
`100`	`digit+`	`<num, 100>`
`)`	右括号模式	`<rparen, _>`

1、词法记号（token）

对词法单元的**分类标签**，记号名 = 记号分类。是编译器内部用来代表这一类词法单元的符号。

内部表示：**<记号名，属性值（可选）>**

- **记号名**：表明它是什么**类别**（如 ID, NUMBER, IF, ASSIGN）。这是**语法分析器**最关心的部分，只看结构，不看具体值。
- **属性值**：一个指向**符号表**中对应条目的**指针**，或者具体的数值。为后期的**语义分析**和**代码生成**保留的具体信息。

  🪄 多个单词匹配同一模式时，提供附加信息。

  <ID,score>、<ID,sum>

记号名分类（通常）：

- 关键字和分隔符一词一类（IF、ELSE、LPAREN、RPAREN等）；
- 标识符（变量）一类（ID）；
- 常数常量按数据类型一类；（NUM、INT_NUM、FLOAT_NUM等)
- 运算符一词一类或者若干个一类（RELOP）

2、模式（pattern）

描述一个词法记号的单词可能具有的形式（如规则），通常用**正则表达式**表示。

👉 一种模式可能有多个字符串可以匹配它

3、词法单元/词素（lexeme）

源程序中匹配模式的字符串序列，是记号的**实例**，**实际存在**的真实代码。。

词法错误

1、**局限性**

词法分析器只有**局部视角**，无法发现结构错误

🪄 把 if 误写为 fi，会被当作合法 ID 传给语法分析器

2、错误发现

仅读入的字符序列**无法和任何一个模式匹配**，才会产生词法错误。

🪄 int **3abc** = 9； 如果当前语言规定 ID 不能以数字开头，则可发现。

a = 12 **@** 3； 如果 @ 不是当前语言里的合法符号，则可发现。

a = ； 属于语法错误，不是词法错误。

3、修复策略

- **紧急方式** (Panic mode)：不断删除剩余输入的前缀字符，直到发现合法记号。

  优点：非常简单，容易实现，而且**绝对不会陷入死循环。**

  缺点：可能会误删掉一些正确的代码，导致后续的语法分析报错（**连环报错**）。
- **错误修补**：对当前剩下的输入字符做一些**小幅度的修改**，试图去猜测程序员原本想写什么。

  常见操作：删除多余字符、插入遗漏字符、替换不正确字符、交换相邻字符。

  优点：保留更多源代码信息

  缺点：实现起来比较复杂；若猜错，可能英法更奇怪的连环错误。

词法记号的描述与识别

串和语言

字母表 → 串 → 语言

1、字母表

有限且**非空**的符号集合（如二进制表、ASCII集），通常用 Σ（Sigma）表示

2、串/符号串

从字母表 Σ 中选取符号，按照一定顺序排列形成的**有限序列。**

串的运算：

- **连接**：把串 y 拼在串 x 的后面，形成新串 xy， 且 εs = sε = s
- **幂**：串的自我连接。$s^0 = ε$，$s^1 = s$，$s^2 = ss$

3、语言

字母表上的一个串的集合。

句子/字：属于该语言的串。

语言的运算：

- 并：$L \cup M = s \mid s \in L$ 或 $s \in M$
- 连接：$LM = st \mid s \in L$ 且 $t \in M$
- 指数/幂：$L^0 = \epsilon$，$L^i = L^{i-1}L$
- 闭包：$L^* = L^0 \cup L^1 \cup L^2 \cup \ldots$ (包含空串)
- **正闭包**：$L^+ = L^1 \cup L^2 \cup \ldots$ (不包含空串，除非 L 本身含空串)

正规式 RE

1、定义

正则表达式，一种特殊表达式，用于表示不太复杂的语言的“含义”（是一种模式）。

不是串，也不是符号，一个正规式 r 表示一种语言 L(r)

正规语言/正规集：由正规式表示的语言。

正规式	正规集	备注
ε	$\{\varepsilon\}$	
a	$\{a\}$	$a \in \sum$
(r)	$L(r)$	r是正规式
$(r) \mid (s)$	$L(r) \cup L(s)$	r和s是正规式
$(r)(s)$	$L(r)L(s)$	r和s是正规式
$(r)^*$	$(L(r))^*$	r是正规式

2、优先级与结合性

`*` **(最高) > 连接 >** `|` **(最低)**；全部为左结合。

3、代数定律

- |：可交换、可结合、和连接可分配
- 连接：可结合、和 | 可分配
- *：幂等性 r** = r*
- r* = (r | ε)* = ε | rr*

正规式定义

为复杂的正规式命名，以便于简写。

$$\text{letter} \rightarrow A \mid B \mid \ldots \mid Z \mid a \mid b \mid \ldots \mid z$$
$$\text{digit} \rightarrow 0 \mid 1 \mid \ldots \mid 9$$
$$\text{id} \rightarrow \text{letter}(\text{letter} \mid \text{digit})^*$$

正规式扩展:

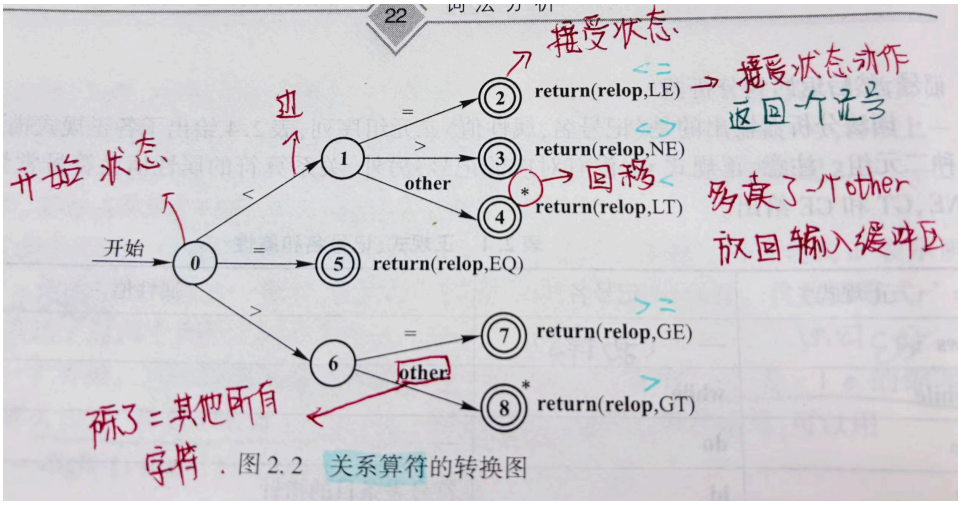
- 一个或多个实例：**+** (即 $r^+ = rr^*$)
- 零个或一个实例：**?** (即 r? = r | ε)
- 字符组：[abc] 表示 a | b | c；[a-z] 表示字母范围。

状态转换图

描绘词法分析器被调用时，为获取下一个记号所做的动作。是手工构造词法分析器的**第一步**。

1、基本组成

- **状态**（节点）、**有向边**（输入字符）
- **初始状态**（识别新token的开始）、接收状态（双圈，完整识别出一个token）
- **动作**（到达接收状态时要执行的代码动作）
- ***** （回移，识别结束时，多读入一个字符，输入指针需要会退一步）



有限自动机 FA

finite automata，识别器的模型化表示，通常用转换图表示

不确定的有限自动机 NFA

1、组成

- 有限的状态集合 S
- 输入符号集合 Σ（不包含 ε）
- 转换函数 move：S x (Σ ∪{ε}) → P(S) （S的幂集）

  move（s,a）：表示从状态s出发，沿着标记为a的边所能到达的**状态集合**
- 状态 S_0 是唯一开始状态
- F ⊆ S是接受状态（终态）集 合

2、特点

- **ε 转换**： NFA 可以拥有标记为 ε 的边

- 同一状态**对同一输入符号**，可能有多条边（转移到多个不同状态）

👉 每当遇到 ε 边 / 同一个字符有多条路可以走 → 试探，回溯，**省空间，速度慢**

3、接受条件

存在**至少一条**从开始状态到**某个接收状态**的路径，其标记拼成输入串

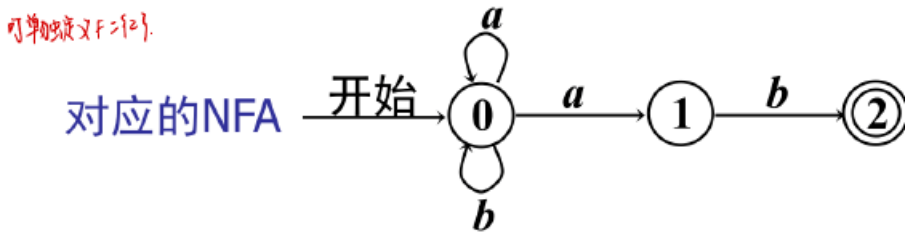
4、转换表

每一栏是一个**状态集合**，无转换时为∅

表中需标注接收状态

状　态	输　入　符　号	
	a	*b*
0	{0, 1}	{0}
1	∅	{2}
2	∅	∅

如果转换函数没有给出对应于某个状态-输入对的信息，就把∅放入相应的表项中



确定的有限自动机 DFA

1、组成

- 有限的状态集合 S
- 输入符号集合 Σ（不包含 ε）
- 转换函数 move：S x (Σ) → P(S) （S的幂集）

  move（s,a）：表示从状态s出发，沿着标记为a的边所能到达的**状态**
- 状态 S_0 是唯一开始状态
- F ⊆ S是接受状态（终态）集合

2、特点

- **没有任何 ε 转换**
- 任何状态对任意输入符号，**最多只有一条**边

3、接受条件

从开始状态起依次读入字符，读完后**刚好到达**某个接受状态（**路径唯一**）

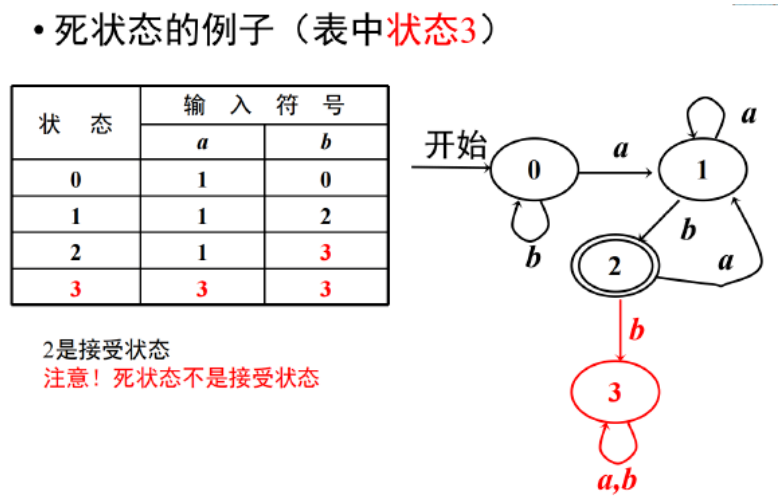
4、转换表

表中每栏至多只有一个状态，无状态的填 `-`

表中需标注接收状态

5、死状态

对某一输入无转换，相当于进入死状态（死状态绝不是接受状态）



6、最长字串匹配原则

当输入串有多个前缀匹配时，总是选择最长的前缀。

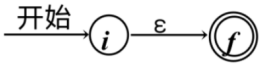
到达终态后只要还有输入，FA就继续前进寻找更长匹配。

从 正规式 到 NFA （Thompson 算法）

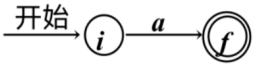
将任何一个正规式（RE）机械化地、按部就班地转换成一个等价的 NFA。

1、基本规则

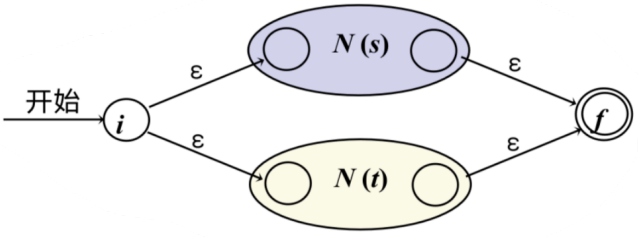
- **识别 ε**



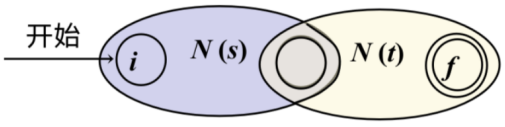
- **识别单个符号 a**



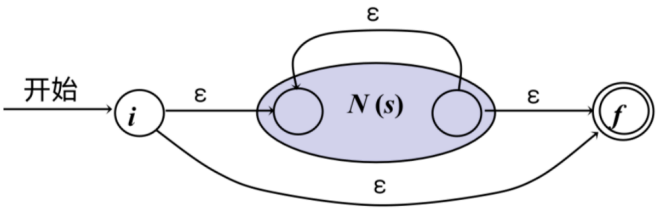
- 识别或运算 s | t



- 识别连接 st

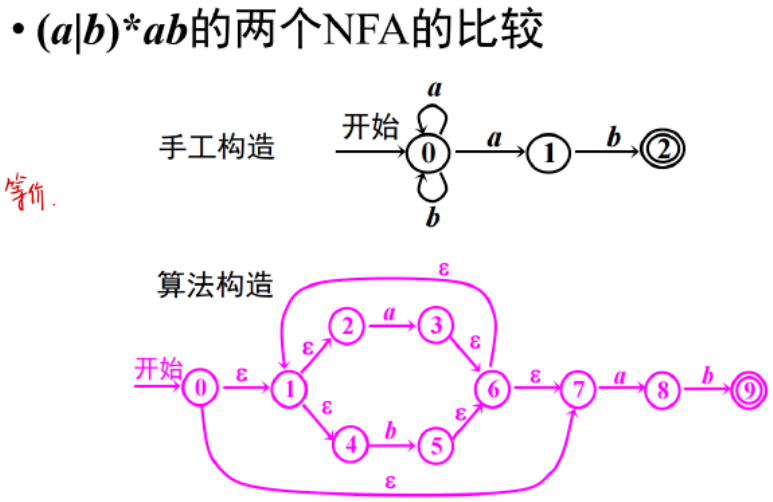


- 识别闭包 s*



2、生成的NFA的性质

- **状态数可控**：状态数最多是正规式 r 中符号和算符**总数的两倍**
- **单头单尾**：只有一个开始状态和一个接收状态，**且接受状态没有向外的转换**
- **边数限制**：每个状态要么有1条符号边，要么最多有2条 ε边指向其它结点



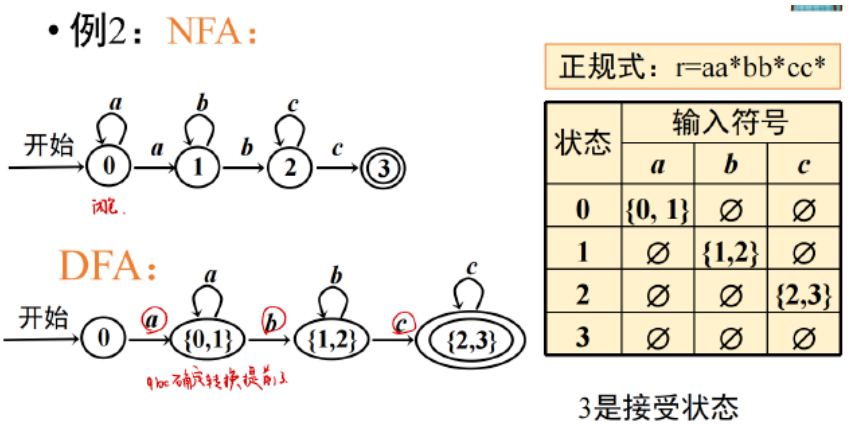
从 NFA 到 DFA（子集构造法）

1、why

- NFA 有 ε 边和多条同名边，容易引起二义性，计算机难以直接模拟（路径太多考虑困难）
- 等价性：对于字母表Σ上的任一 NFA，必存在 Σ 上与其等价的 DFA
- NFA中接受一个串的路径往往有多条,考虑起来比较困难

2、核心思路

DFA 的一个状态，实际上是 NFA 多个状态集合。



正规式：r=aa*bb*cc*

状态	输入符号		
	a	b	c
0	{0, 1}	∅	∅
1	∅	{1,2}	∅
2	∅	∅	{2,3}
3	∅	∅	∅

3是接受状态

3、集合操作

- ε-closure(S)：从**单个状态** S 出发，只走 ε 边能到达的所有状态集合

- ε-closure(T)：从**状态集合** T 中的所有状态出发，只走 ε 边能到达的集合
- move(T, a)：从状态集合T 出发，经过刚好一条 a 边能到达的集合
  通常有ε-closure(move(T, a))

❗ ε-闭包**永远包含状态自己**

4、算法流程

- **确定 DFA 初始状态 A**：计算 NFA 初始状态 S_0 的 ε-闭包
- **循环遍历**：遍历每一个输入符号 a，计算 `U = ε-closure(move(T, a))`

  如果 U 是一个新集合，将其作为未标记状态加入DFA状态集

  直到没有新状态
- **标记 DFA 接受状态**：包含原 NFA 接受状态的

👉 **子集构造法生成的DFA 不一定是最简的**

从 DFA 到 最简DFA（极小化DFA状态数算法）

1、why

子集构造法得到的DFA状态太多，会导致存放状态转移表的内存占用越大

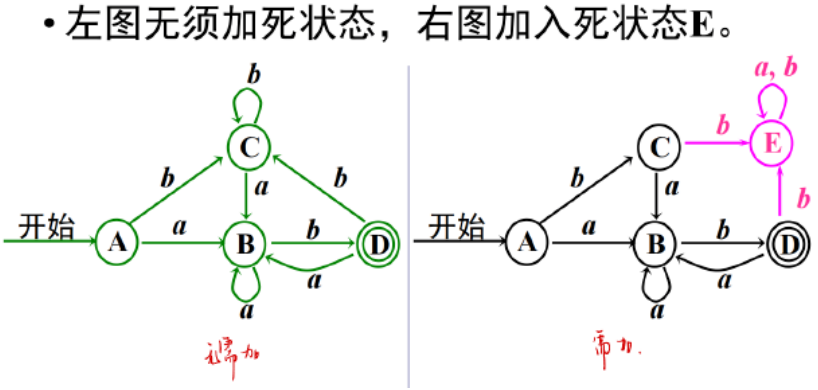
2、满足条件

- **没有多余状态**（死状态/不可达状态）：从初始状态出发根本走不到的状态，或者走进去就永远出不来且不是终态的“黑洞”状态，统统删掉。
- **没有等价状态**：如果两个状态在逻辑上起的作用**完全一模一样**，就应该把它们合并成一个状态。

3、基本概念

- **可区别状态**：存在一个输入串 w，使得从状态 s 和 t 出发，最终一个停留在**接受状态**，另一个停留在**非接受状态**
- **理论唯一性**：不计同构，每个正规集都可以由一个**状态数最少**的 DFA **唯一识别**
- **算法前提**：转换函数必须是全函数（任何状态对任何输入都有转移）

  若不是全函数，需引入**死状态**补充



4、算法流程

- **初始划分** Π_0：接受状态子集 F、非接受状态子集 S-F
- **迭代划分**：对当前 Π 中诶个子集 G 进行检查，若对于任意输入符号 a，G 中状态转移到了不同的子集中，则将状态分割到新的不同子集中

- **终止条件**：新划分 Π_{new}与旧划分 Π 相同（即没有任何子集可以继续划分）
- **选取代表与重构**：在最终的每个状态子集中选一个状态作为代表（即最简DFA的新状态），并修改状态表
    - 新开始状态：原包含 S_0 的子集
    - 新接受状态：原属于 F 的子集